

Báo cáo tháng 10

1 Giới thiệu

Dựa trên bài báo gốc “*Optimization of time and energy in straight one-sided robotic assembly*” (2025) của Rehab Seif ElMolouk, Amin M. K. El-Kharbotly và Raghda B. Taha. Mô hình giải được tham khảo từ phương pháp mã hóa trong bài báo “*Simple Assembly Line Balancing Problem with Power Peak Minimization*”.

2 Mô hình bài toán

2.1 Định nghĩa các tham số

- N_a : số công việc.
- N_w : số trạm làm việc.
- N_r : số robot.
- $T[j][r]$: thời gian robot r cần để hoàn thành công việc j .
- $EP[r]$: năng lượng tiêu thụ của robot r trên một đơn vị thời gian.
- $graph[j]$: danh sách các công việc kế tiếp của công việc j .

2.2 Công thức giải

Công thức (1) được cải tiến bằng cách sử dụng *staircase encoding* để loại bỏ các biểu diễn *backward* và *forward encoding*. Các công thức từ (2) đến (15) được giữ nguyên như mô hình ban đầu.

3 Tổng quan mô hình giải thuật

3.1 Phân đọc dữ liệu

- Đọc dữ liệu đầu vào.
- Từ dữ liệu đầu vào, thu được các giá trị N_a , N_r , N_w cùng tập giá trị T , EP và $graph$.

3.2 Giai đoạn Preprocess

- Khởi tạo tạm thời chu kỳ CT bằng tổng thời gian của n công việc có thời gian dài nhất, trong đó n là số công việc trung bình trên mỗi trạm.
- Tính toán tập $ip1[j][k]$: cho biết công việc j có thể thực hiện trên trạm k hay không.
- Tính toán tập $ip2[j][k][t]$: cho biết công việc j có thể thực hiện trên trạm k tại thời điểm t hay không.

3.3 Giai đoạn Fixed Clauses

- Bao gồm các công thức từ (1) đến (12), được khởi tạo cố định theo giá trị CT ban đầu trong giai đoạn **Preprocess**.
- Hàm này chỉ được gọi một lần và các công thức sẽ được giữ nguyên trong suốt quá trình giải.

3.4 Giai đoạn Dynamic Clauses

- Bao gồm các công thức (13) giới hạn CT , (14) giới hạn năng lượng và (15) loại bỏ nghiệm trùng lặp.
- Hàm này được gọi lại sau mỗi lần lặp do cần cập nhật trọng số và giá trị mục tiêu.

3.5 Bộ giải (Solver)

- Khởi tạo giá trị $LB = 0$ và $UB =$ giá trị z_value lớn nhất có thể.
- Mục tiêu: tối thiểu hoá $z_value = w_1 \times CT + w_2 \times Energy$.
- Sử dụng phương pháp *binary search*:
 - Sau mỗi vòng lặp, nhận được các giá trị $actual_ct$, $actual_e$ và nghiệm tương ứng.
 - Tính toán, so sánh z_value , và cập nhật kết quả tốt nhất.
 - Tiếp tục lặp cho đến khi không thể tìm thêm nghiệm khả thi.
- Trả về nghiệm và kết quả cuối cùng.

4 So sánh với cài đặt trước đây

- Cài đặt cũ khởi tạo toàn bộ các *clauses* mỗi khi có giá trị CT mới, dẫn đến số lần khởi tạo quá nhiều.
- Cài đặt cũ chỉ sử dụng tối ưu thời gian ($BestCT - 1$) trong mỗi lần lặp, không áp dụng tìm kiếm nhị phân.
- Phiên bản cải tiến tách biệt các *clauses* cố định và động, giúp giảm đáng kể thời gian khởi tạo.
- Thời gian thực thi giảm khoảng một nửa so với bản cũ, mặc dù vẫn chậm hơn các phương pháp ILP.

5 Hướng phát triển tiếp theo

- Giảm giá trị CT khởi tạo ban đầu.
- Tìm kiếm phương pháp *encoding* mới để giảm số lần lặp khi khởi tạo.
- Chia bài toán thành hai mục tiêu riêng: tối thiểu hoá CT và tối thiểu hoá $Energy$.